Projeto: Rede Social Simples
Disciplina: Algoritmos, Estruturas de Dados e Programação

Autores:
- Gustavo Soares Lima Rios
- Pedro Malcon Cabral Silva
- Vinícius Fernandes Vieira

Organização do Projeto
O sistema foi desenvolvido de forma modular, separando responsabilidades entre diferentes arquivos para facilitar manutenção, testes e reutilização de código.

Código-Fonte Completo:

database.py
```
# Camada de acesso ao banco de dados.

# Contém todas as operações de leitura e escrita no SQLite.
# É o arquivo mais importante do projeto: centraliza toda a lógica de persistência.

import sqlite3
from usuario import Usuario

# Nome do arquivo do banco — deve ser o mesmo definido em init_db.py
DB_NAME = "rede_social.db"


def conectar():
    # Abre e retorna uma conexão com o banco de dados
    return sqlite3.connect(DB_NAME)


def adicionar_usuario(username, nome, idade):
    conn = conectar()
    cursor = conn.cursor()

    try:
        # Tenta inserir o novo usuário no banco
        # O SQLite rejeita automaticamente se o username já existir (UNIQUE)
        cursor.execute(
            """
            INSERT INTO usuarios
            (username, nome, idade)
            VALUES (?, ?, ?)
```

```
            """,
            (username, nome, idade)
        )

        conn.commit()
        return True  # Inserção bem-sucedida

    except sqlite3.IntegrityError:
        # Erro disparado quando o username já existe no banco
        return False

    finally:
        # Garante que a conexão seja fechada mesmo em caso de erro
        conn.close()


def remover_usuario(username):
    conn = conectar()
    cursor = conn.cursor()

    # Busca o ID do usuário pelo username
    cursor.execute(
        """
        SELECT id
        FROM usuarios
        WHERE username = ?
        """,
        (username,)
    )

    usuario = cursor.fetchone()

    # Se não encontrou, encerra sem fazer nada
    if not usuario:
        conn.close()
        return False

    user_id = usuario[0]

    # Remove todas as amizades do usuário antes de removê-lo
    # Necessário para não deixar registros órfãos na tabela de amizades
    cursor.execute(
        """
        DELETE FROM amizades
```

```
        WHERE usuario1 = ?
          OR usuario2 = ?
        """,
        (user_id, user_id)
    )

    # Agora remove o próprio usuário
    cursor.execute(
        """
        DELETE FROM usuarios
        WHERE id = ?
        """,
        (user_id,)
    )

    conn.commit()
    conn.close()

    return True


def listar_usuarios():
    conn = conectar()
    cursor = conn.cursor()

    # Busca todos os usuários ordenados por username
    cursor.execute(
        """
        SELECT id, username, nome, idade
        FROM usuarios
        ORDER BY username
        """
    )

    # Converte cada linha do resultado em um objeto Usuario
    usuarios = [
        Usuario(*dados)
        for dados in cursor.fetchall()
    ]

    conn.close()

    return usuarios
```

```
def obter_id(username):
    # Função auxiliar: retorna o ID de um usuário pelo username
    # Usada internamente por criar_amizade e remover_amizade
    conn = conectar()
    cursor = conn.cursor()

    cursor.execute(
        """
        SELECT id
        FROM usuarios
        WHERE username = ?
        """,
        (username,)
    )

    resultado = cursor.fetchone()

    conn.close()

    if resultado:
        return resultado[0]

    return None  # Usuário não encontrado


def criar_amizade(username1, username2):
    # Impede que um usuário se adicione como amigo de si mesmo
    if username1 == username2:
        return False

    # Obtém os IDs dos dois usuários
    id1 = obter_id(username1)
    id2 = obter_id(username2)

    # Se qualquer um dos usernames não existir, cancela
    if id1 is None or id2 is None:
        return False

    conn = conectar()
    cursor = conn.cursor()

    # Verifica se a amizade já existe, checando as duas ordens possíveis
    # (id1, id2) ou (id2, id1) — amizade é bidirecional
```

```
    cursor.execute(
        """
        SELECT *
        FROM amizades
        WHERE
        (usuario1 = ? AND usuario2 = ?)
        OR
        (usuario1 = ? AND usuario2 = ?)
        """,
        (id1, id2, id2, id1)
    )

    # Se já existe amizade, não cria duplicata
    if cursor.fetchone():
        conn.close()
        return False

    # Insere a nova amizade no banco
    cursor.execute(
        """
        INSERT INTO amizades
        (usuario1, usuario2)
        VALUES (?, ?)
        """,
        (id1, id2)
    )

    conn.commit()
    conn.close()

    return True


def remover_amizade(username1, username2):
    # Obtém os IDs dos dois usuários
    id1 = obter_id(username1)
    id2 = obter_id(username2)

    # Se qualquer um não existir, não há o que remover
    if id1 is None or id2 is None:
        return False

    conn = conectar()
    cursor = conn.cursor()
```

```
    # Remove a amizade checando as duas ordens possíveis
    cursor.execute(
        """
        DELETE FROM amizades
        WHERE
        (usuario1 = ? AND usuario2 = ?)
        OR
        (usuario1 = ? AND usuario2 = ?)
        """,
        (id1, id2, id2, id1)
    )

    conn.commit()

    # rowcount indica quantas linhas foram afetadas
    # Se for 0, a amizade não existia
    removidas = cursor.rowcount

    conn.close()

    return removidas > 0


def listar_amigos(username):
    user_id = obter_id(username)

    # Se o usuário não existe, retorna lista vazia
    if user_id is None:
        return []

    conn = conectar()
    cursor = conn.cursor()

    # Busca todos os usuários que têm amizade com user_id
    # O JOIN cruza a tabela usuarios com amizades considerando as duas ordens
    cursor.execute(
        """
        SELECT u.id,
               u.username,
               u.nome,
               u.idade
        FROM usuarios u
        JOIN amizades a
```

```
        ON (
            (a.usuario1 = ? AND a.usuario2 = u.id)
            OR
            (a.usuario2 = ? AND a.usuario1 = u.id)
        )
        """,
        (user_id, user_id)
    )

    # Converte cada resultado em um objeto Usuario
    amigos = [
        Usuario(*dados)
        for dados in cursor.fetchall()
    ]

    conn.close()

    return amigos


def contar_amigos(user_id):
    conn = conectar()
    cursor = conn.cursor()

    # Conta quantas amizades existem envolvendo esse user_id
    # em qualquer uma das colunas (usuario1 ou usuario2)
    cursor.execute(
        """
        SELECT COUNT(*)
        FROM amizades
        WHERE usuario1 = ?
           OR usuario2 = ?
        """,
        (user_id, user_id)
    )

    quantidade = cursor.fetchone()[0]

    conn.close()

    return quantidade
```

---

usuario.py

```
# Responsável por representar um usuário como objeto Python.
# Utilizado pelo database.py para retornar dados estruturados ao invés de tuplas brutas.

class Usuario:
    # Construtor: recebe os dados vindos do banco e os armazena como atributos
    def __init__(self, user_id, username, nome, idade):
        self.id = user_id        # ID gerado automaticamente pelo SQLite
        self.username = username  # Identificador único do usuário (ex: joao123)
        self.nome = nome          # Nome completo (ex: João Silva)
        self.idade = idade        # Idade em anos

    # Representação textual do objeto, usada ao imprimir um usuário diretamente
    def __str__(self):
        return (
            f"ID: {self.id} | "
            f"Username: {self.username} | "
            f"Nome: {self.nome} | "
            f"Idade: {self.idade}"
        )
```

---

rede.py

```
# Camada intermediária entre a interface (main.py) e o banco de dados (database.py).
# Mantida principalmente para atender ao requisito de modularização:
# main.py nunca acessa database.py diretamente, sempre passa por RedeSocial.

import database


class RedeSocial:

    def adicionar_usuario(self, username, nome, idade):
        # Repassa o cadastro para o banco e retorna True/False
        return database.adicionar_usuario(username, nome, idade)

    def remover_usuario(self, username):
        # Remove o usuário e todas as suas amizades do banco
        return database.remover_usuario(username)

    def listar_usuarios(self):
        # Retorna uma lista de objetos Usuario com todos os cadastrados
        return database.listar_usuarios()
```

```
    def criar_amizade(self, username1, username2):
        # Cria vínculo entre dois usuários; retorna False se inválido ou duplicado
        return database.criar_amizade(username1, username2)

    def remover_amizade(self, username1, username2):
        # Remove o vínculo entre dois usuários; retorna False se não existia
        return database.remover_amizade(username1, username2)

    def listar_amigos(self, username):
        # Retorna lista de objetos Usuario que são amigos do username informado
        return database.listar_amigos(username)

    def contar_amigos(self, user_id):
        # Retorna a quantidade de amigos de um usuário pelo seu ID
        return database.contar_amigos(user_id)
```

---

init_db.py

```
# Responsável por criar o banco de dados e as tabelas necessárias.
# Deve ser executado uma única vez antes de usar o sistema,
# mas pode ser chamado a qualquer momento pois usa IF NOT EXISTS.

import sqlite3

# Nome do arquivo do banco de dados SQLite
DB_NAME = "rede_social.db"


def criar_banco():
    # Abre (ou cria) o arquivo do banco de dados
    conn = sqlite3.connect(DB_NAME)
    cursor = conn.cursor()

    # Cria a tabela de usuários, caso ainda não exista
    # O id é gerado automaticamente pelo SQLite (AUTOINCREMENT)
    # O username é único: o banco rejeita duplicatas automaticamente
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS usuarios (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT UNIQUE NOT NULL,
            nome TEXT NOT NULL,
            idade INTEGER NOT NULL
        )
```

```
    """)

    # Cria a tabela de amizades, caso ainda não exista
    # Cada linha representa um vínculo entre dois usuários (pelos seus IDs)
    # As FOREIGN KEYs garantem que os IDs referenciam usuários existentes
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS amizades (
            usuario1 INTEGER NOT NULL,
            usuario2 INTEGER NOT NULL,

            FOREIGN KEY (usuario1)
                REFERENCES usuarios(id),

            FOREIGN KEY (usuario2)
                REFERENCES usuarios(id)
        )
    """)

    # Salva as alterações e fecha a conexão
    conn.commit()
    conn.close()

    print("Banco criado com sucesso!")


# Permite executar diretamente: python init_db.py
if __name__ == "__main__":
    criar_banco()
```

---

popular_banco.py

```
# Arquivo auxiliar para popular o banco com dados de exemplo.
# Usado para demonstração na apresentação.
# Pode ser executado múltiplas vezes: limpa os dados anteriores antes de inserir.

from rede import RedeSocial
from init_db import criar_banco
import database
import sqlite3


def limpar_banco():
    # Remove todos os registros das tabelas e reseta o contador de IDs
    # Necessário para evitar conflitos ao rodar o script mais de uma vez
```

```
    conn = database.conectar()
    cursor = conn.cursor()
    cursor.execute("DELETE FROM amizades")
    cursor.execute("DELETE FROM usuarios")
    cursor.execute("DELETE FROM sqlite_sequence WHERE name='usuarios'")
    conn.commit()
    conn.close()


def main():
    # Garante que o banco existe antes de tentar popular
    criar_banco()
    limpar_banco()

    rede = RedeSocial()

    # Lista de usuários de exemplo: (username, nome, idade)
    usuarios = [
        ("joao123", "João Silva", 20),
        ("maria456", "Maria Souza", 19),
        ("ana789", "Ana Lima", 21),
        ("carlos10", "Carlos Oliveira", 22),
        ("pedro22", "Pedro Santos", 18)
    ]

    for username, nome, idade in usuarios:
        rede.adicionar_usuario(username, nome, idade)

    # Lista de amizades de exemplo: (username1, username2)
    amizades = [
        ("joao123", "maria456"),
        ("joao123", "ana789"),
        ("maria456", "carlos10"),
        ("pedro22", "joao123")
    ]

    for user1, user2 in amizades:
        rede.criar_amizade(user1, user2)

    print("Banco populado com sucesso!")


if __name__ == "__main__":
    main()
```

teste_sistema.py

```
# Arquivo auxiliar de testes automatizados.
# Valida as principais funcionalidades do sistema de forma rápida.
# Útil para verificar que tudo funciona antes da apresentação.

from rede import RedeSocial
from init_db import criar_banco


# Contadores globais de testes aprovados e reprovados
ok = 0
falhas = 0


def checar(descricao, condicao):
    # Avalia uma condição e registra o resultado do teste
    global ok, falhas

    if condicao:
        ok += 1
        print(f"✓ {descricao}")
    else:
        falhas += 1
        print(f"✗ {descricao}")


def titulo(texto):
    print("\n" + "=" * 50)
    print(texto)
    print("=" * 50)


def main():
    global ok, falhas

    # Garante que o banco existe antes de rodar os testes
    criar_banco()

    rede = RedeSocial()

    titulo("TESTES DO SISTEMA")

    # Usernames usados apenas durante os testes
```

```
    usuario_teste = "teste_usuario"
    amigo_teste = "teste_amigo"

    # Teste 1: cadastrar um novo usuário
    resultado = rede.adicionar_usuario(usuario_teste, "Usuário Teste", 20)
    checar("Cadastro de usuário", resultado)

    # Teste 2: tentar cadastrar o mesmo username de novo (deve falhar)
    resultado = rede.adicionar_usuario(usuario_teste, "Outro Usuário", 30)
    checar("Bloqueio de username duplicado", not resultado)

    # Cria um segundo usuário para testar as amizades
    rede.adicionar_usuario(amigo_teste, "Amigo Teste", 21)

    # Teste 3: criar amizade entre os dois usuários de teste
    resultado = rede.criar_amizade(usuario_teste, amigo_teste)
    checar("Criar amizade", resultado)

    # Teste 4: tentar criar amizade consigo mesmo (deve falhar)
    resultado = rede.criar_amizade(usuario_teste, usuario_teste)
    checar("Bloqueio de autoamizade", not resultado)

    # Teste 5: verificar se a listagem de amigos retorna resultados
    amigos = rede.listar_amigos(usuario_teste)
    checar("Listar amigos", len(amigos) > 0)

    # Teste 6: remover a amizade criada
    resultado = rede.remover_amizade(usuario_teste, amigo_teste)
    checar("Remover amizade", resultado)

    # Teste 7: remover o usuário principal de teste
    resultado = rede.remover_usuario(usuario_teste)
    checar("Remover usuário", resultado)

    # Limpa também o segundo usuário criado durante os testes
    rede.remover_usuario(amigo_teste)

    # Exibe o resultado final
    print("\n" + "=" * 50)
    print(f"Resultado: {ok} passaram | {falhas} falharam")
    print("=" * 50)


if __name__ == "__main__":
```

```
main()
```

Código-Fonte Completo

main.py

```
# Interface textual do sistema, estilizada com caracteres de desenho de caixa
(box-drawing).
# Responsável pelos menus, entrada de dados e chamadas às funções da RedeSocial.
# É o ponto de entrada do programa: python main.py

import sys
from rede import RedeSocial
from init_db import criar_banco

# Força a saída em UTF-8, garantindo que os caracteres especiais
# (═, ║, ╔, etc.) sejam exibidos corretamente em qualquer terminal,
# incluindo o cmd.exe do Windows, que por padrão usa outra codificação.
sys.stdout.reconfigure(encoding='utf-8')

LARGURA = 60


def linha_da_caixa(texto, borda):
    # Monta uma linha de conteúdo dentro da caixa, cortando o texto
    # se ele for maior que a largura disponível, para não desalinhar
    # a borda direita
    espaco_disponivel = LARGURA - 4

    if len(texto) > espaco_disponivel:
        texto = texto[:espaco_disponivel - 3] + "..."

    print(f"{borda} " + texto.ljust(espaco_disponivel) + f" {borda}")


def caixa_dupla(titulo_texto, linhas):
    # Desenha uma caixa com bordas duplas (═ ║ ╔ ╗ ╚ ╝ ╠ ╣)
    # Usada no menu principal, para dar destaque
    print("╔" + "═" * (LARGURA - 2) + "╗")
    print("║" + titulo_texto.center(LARGURA - 2) + "║")
    print("╠" + "═" * (LARGURA - 2) + "╣")

    for linha_texto in linhas:
        linha_da_caixa(linha_texto, "║")

    print("╚" + "═" * (LARGURA - 2) + "╝")
```

```
def caixa_simples(titulo_texto):
    # Desenha um cabeçalho com bordas simples (─ │ ┌┐ └┘)
    # Usado nas telas de listagem (perfis, amigos)
    print(" ┌" + "─" * (LARGURA - 2) + "┐ ")
    print(" │ " + titulo_texto.center(LARGURA - 2) + " │ ")
    print(" ├" + "─" * (LARGURA - 2) + "┤ ")


def fechar_caixa_simples():
    print(" └" + "─" * (LARGURA - 2) + "┘ ")


def pausar():
    input("\nPressione Enter para continuar...")


def menu():
    opcoes = [
        "[1] Adicionar usuário",
        "[2] Remover usuário",
        "[3] Criar amizade",
        "[4] Remover amizade",
        "[5] Listar amizades",
        "[6] Listar perfis",
        "[7] Sair",
    ]
    caixa_dupla("REDE SOCIAL", opcoes)


def adicionar_usuario(rede):
    username = input("Username: ").strip()
    nome = input("Nome: ").strip()

    # Valida se a idade é um número inteiro
    try:
        idade = int(input("Idade: "))
    except ValueError:
        print("Idade inválida.")
        return

    # Campos obrigatórios não podem estar vazios
    if not username or not nome:
        print("Username e nome são obrigatórios.")
        return

    if rede.adicionar_usuario(username, nome, idade):
        print("Usuário cadastrado com sucesso!")
    else:
```

```
        # Falha ocorre quando o username já existe no banco
        print("Username já existe.")


def remover_usuario(rede):
    username = input("Username do usuário: ").strip()

    if rede.remover_usuario(username):
        print("Usuário removido.")
    else:
        print("Usuário não encontrado.")


def criar_amizade(rede):
    username1 = input("Username 1: ").strip()
    username2 = input("Username 2: ").strip()

    if rede.criar_amizade(username1, username2):
        print("Amizade criada.")
    else:
        # Pode falhar por: usuário inexistente, autoamizade ou amizade duplicada
        print("Não foi possível criar amizade.")


def remover_amizade(rede):
    username1 = input("Username 1: ").strip()
    username2 = input("Username 2: ").strip()

    if rede.remover_amizade(username1, username2):
        print("Amizade removida.")
    else:
        print("Amizade não encontrada.")


def listar_amigos(rede):
    username = input("Username: ").strip()

    amigos = rede.listar_amigos(username)

    caixa_simples("AMIGOS")

    if not amigos:
        linha_da_caixa("Nenhum amigo encontrado.", "│")
    else:
        # Exibe username e nome de cada amigo
        for amigo in amigos:
            texto = f"{amigo.username} | {amigo.nome}"
            linha_da_caixa(texto, "│")
```

```
    fechar_caixa_simples()


def listar_perfis(rede):
    usuarios = rede.listar_usuarios()

    caixa_simples("PERFIS")

    if not usuarios:
        linha_da_caixa("Nenhum usuário cadastrado.", "│")
    else:
        # Para cada usuário, busca a quantidade de amigos pelo ID
        for usuario in usuarios:
            quantidade = rede.contar_amigos(usuario.id)

            texto = (
                f"ID:{usuario.id} {usuario.username} "
                f"{usuario.nome} {usuario.idade}a "
                f"Amigos:{quantidade}"
            )
            linha_da_caixa(texto, "│")

    fechar_caixa_simples()


def main():
    # Garante que o banco e as tabelas existam antes de qualquer operação
    # Usa CREATE TABLE IF NOT EXISTS, então é seguro chamar sempre
    criar_banco()

    rede = RedeSocial()

    while True:
        menu()

        opcao = input("Escolha uma opção: ").strip()
        print()

        # match/case: estrutura moderna do Python 3.10+ para tratar múltiplas opções
        match opcao:

            case "1":
                adicionar_usuario(rede)

            case "2":
                remover_usuario(rede)
```

```
            case "3":
                criar_amizade(rede)

            case "4":
                remover_amizade(rede)

            case "5":
                listar_amigos(rede)

            case "6":
                listar_perfis(rede)

            case "7":
                print("Encerrando...")
                break

            case _:
                print("Opção inválida.")

        # Pausa entre operações para o usuário ler o resultado
        pausar()


if __name__ == "__main__":
    main()
```